

RICM4/PS 2014: Images Panini

Arnaud Legrand

14/10/2014

Contents

Approche expérimentale	2
Q1) Simulation	2
Q2) Étude	2
Modélisation	6
Q4) Calculs des lois de probabilités	6
Synthèse	7
Q5) Vérification de la cohérence des calculs	7
Q6) Et mon petit cousin ?	8
Extension au cas non uniforme	8
Q7) Simulation du marchand malhonnête	8
Q8) Le cas d'une distribution exponentielle	12



L'objectif de ce devoir était d'écrire une petite simulation autour du célèbre problème du coupon collector

Votre petit cousin collectionne les vignettes autocollantes des footballeurs célèbres. Il achète au

marchand de journaux des pochettes de 10 vignettes à un prix de 2 euros. L'album de la collection complète contient $M = 300$ places distinctes pour les vignettes autocollantes (prix d'achat de l'album 4 euros). Il souhaite savoir combien de pochettes il devra acheter pour avoir toute la collection. Saurez vous lui répondre ?

Dans un premier temps, on simplifie le problème en supposant que l'on achète les vignettes une par une et on note T le nombre total de vignettes achetées pour avoir toute la collection. À la fin, il restera plein de doubles ! On supposera également que l'éditeur des vignettes est honnête et qu'il imprime les vignettes dans les mêmes proportions (il n'y a pas de vignettes plus rares que d'autres).

Approche expérimentale

Q1) Simulation

Écrire un simulateur en R générant T en fonction de M .

Commençons par écrire la fonction qui permet de simuler le remplissage de l'album et au bout de combien de temps la collection est terminée.

```
library(ggplot2)
```

```
## Registered S3 methods overwritten by 'ggplot2':  
##   method      from  
##   [.quosures  rlang  
##   c.quosures  rlang  
##   print.quosures rlang
```

```
library(plyr)  
set.seed(42);  
  
new_collection <- function(album_size=100) {  
  album=rep_len(0, album_size)  
  nb_images = 0;  
  time = 0;  
  while(nb_images<album_size) {  
    time = time + 1  
    idx = floor(runif(1,1, album_size+1))  
    if(album[idx]==0) {  
      album[idx] = 1  
      nb_images = nb_images + 1  
    }  
  }  
  time  
}
```

Q2) Étude

Étudier expérimentalement la valeur moyenne de T en fonction de M . On prendra des échantillons de l'ordre de 100 ou 1000 et une dizaine de valeurs M entre 10 et 200. Une représentation graphique sera appréciée... Que remarquez vous? ### Génération d'un ensemble de populations Réalisons maintenant un ensemble de simulations et rassemblons tout ceci dans une data-frame. Pour chaque taille d'album entre 10 et 200 par pas de 20, je génère 100 scénarios différents et je calcule combien de temps a été nécessaire pour compléter l'album.

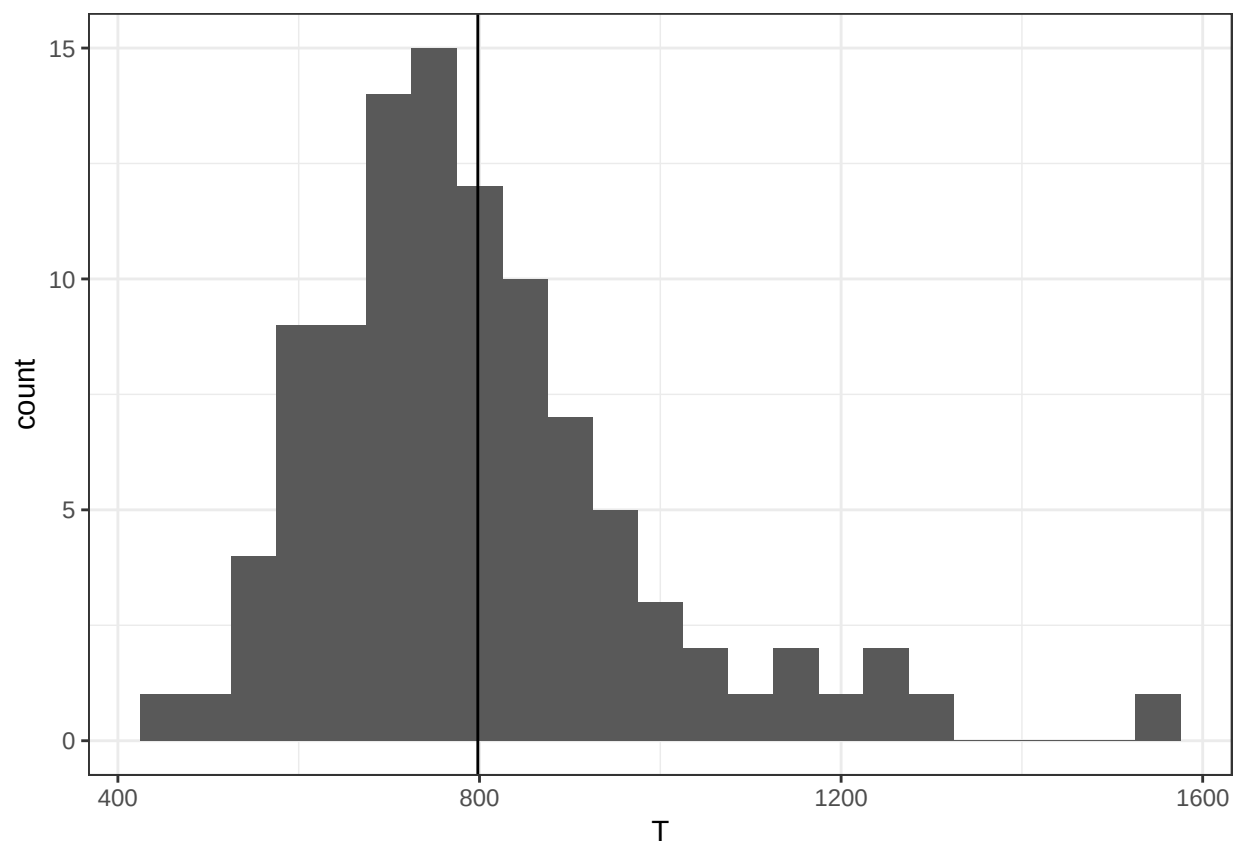
```
df=data.frame(M=c(),T=c())
for(M in seq(10,200,20)) {
  for(samples in 1:100) {
    df = rbind(df,data.frame(M=c(M),T=new_collection(M)))
  }
}
```

Lors de nos investigations initiales, nous avons choisi un plus grand nombre de scénarios et un échantillonnage plus fin du nombre d'images. Les temps de calculs étaient alors assez longs sans que les tendances soient plus claires pour autant. Le choix précédent nous a donc semblé un compromis raisonnable.

Étude de $E(T)$

Regardons la distribution de nos échantillons pour l'album de taille 150.

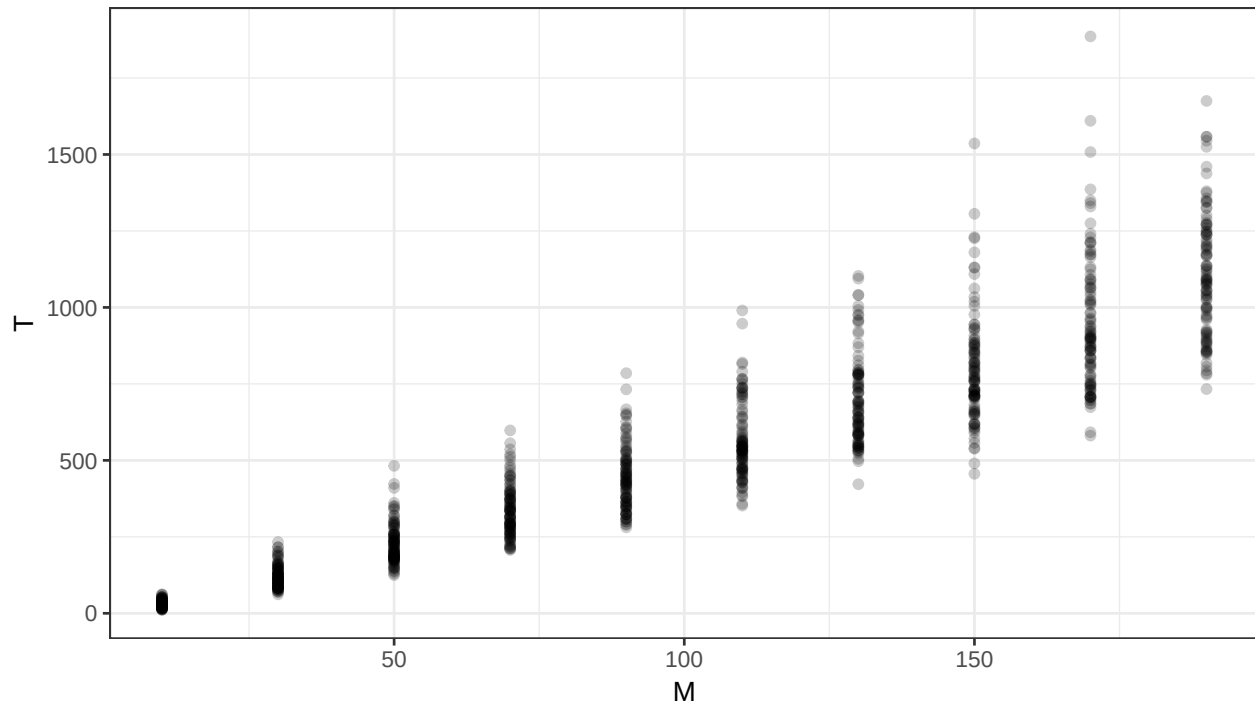
```
ggplot(data=df[df$M==150,], aes(x=T)) + geom_histogram(binwidth=50) + theme_bw() + geom_vline(xintercept=
```



La variabilité est assez importante. La distribution a l'air relativement sympa (un seul mode). Comme on pouvait s'y attendre, c'est non symétrique et pas mal tirée vers la droite (skew). Pas clair qu'une centaine d'échantillons soit suffisant pour avoir une bonne estimation de l'espérance.

Faisons une première visualisation de notre échantillon:

```
ggplot(data=df, aes(x=M,y=T)) + geom_point(alpha=.2) + theme_bw()
```



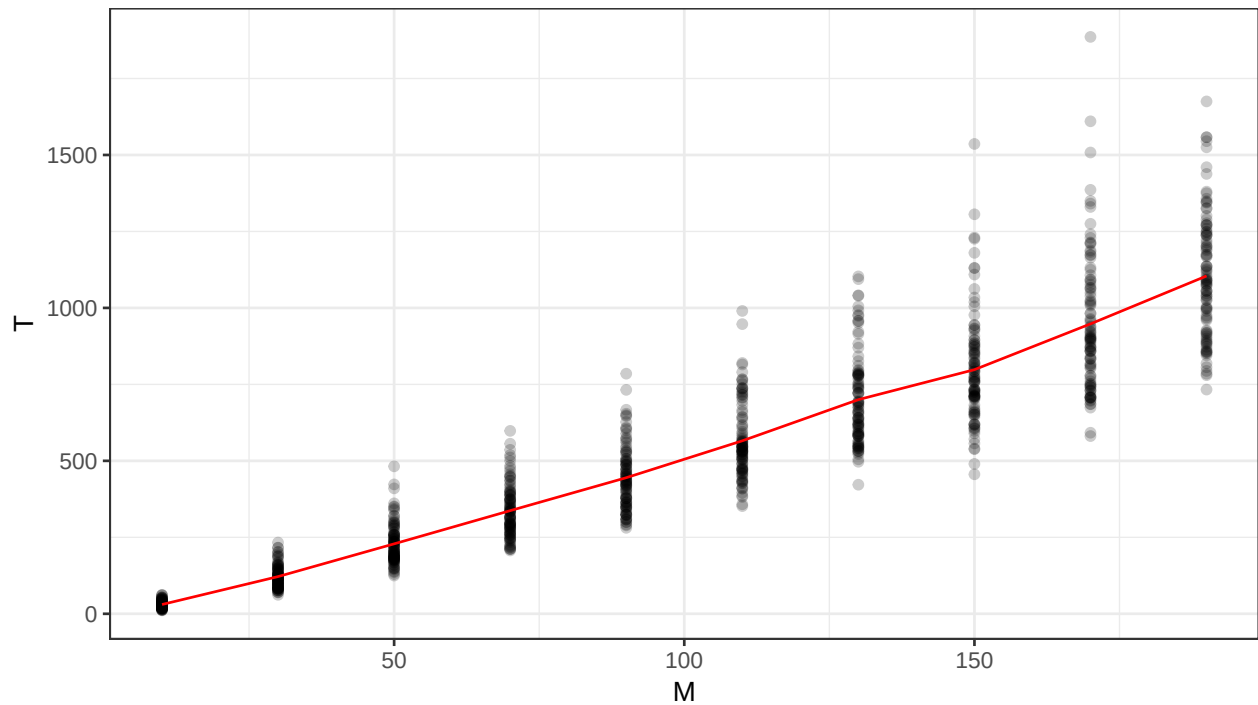
Comme on pouvait s'y attendre, le temps moyen nécessaire à collecter augmente avec le nombre d'images. La variabilité augmente également énormément et il serait probablement plus judicieux de considérer plus de scénarios quand le nombre d'images augmente. Calculons donc la moyenne et l'écart type pour chacune des tailles possibles:

```
df_stat = ddpily(df,c("M"),summarise,mean=mean(T),var=var(T),se=2*sqrt(var(T))/sqrt(length(T)))
df_stat
```

##	M	mean	var	se
## 1	10	30.41	142.9514	2.391246
## 2	30	121.68	1378.3006	7.425094
## 3	50	228.70	4315.3030	13.138193
## 4	70	337.24	7233.0529	17.009471
## 5	90	444.92	10938.0945	20.917069
## 6	110	565.59	15527.4565	24.921843
## 7	130	700.11	22078.4423	29.717633
## 8	150	798.14	32079.7378	35.821635
## 9	170	947.80	50689.2121	45.028530
## 10	190	1104.44	40483.9661	40.241255

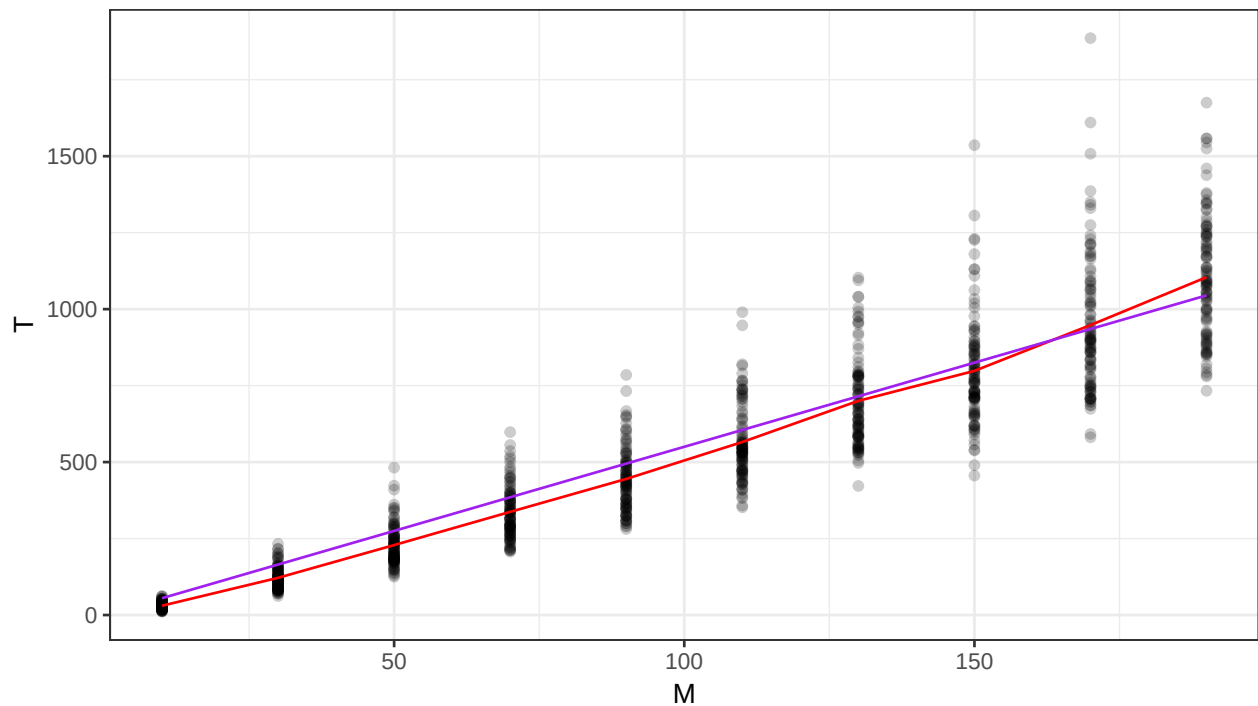
Puis ajoutons cette moyenne (en rouge) sur la courbe précédente.

```
ggplot(data=df, aes(x=M,y=T)) + geom_point(alpha=.2) +
  geom_line(data=df_stat,aes(y=mean),color="red") +
  theme_bw()
```



Tiens, on dirait que c'est linéaire en fait... Qu'est-ce que ça donne si j'essaye de coller une droite ?

```
E_T <- function(x) {x*5.5}
ggplot(data=df, aes(x=M,y=T)) + geom_point(alpha=.2) +
  geom_line(data=df_stat,aes(y=mean),color="red") +
  stat_function(fun = E_T, color="purple") +
  theme_bw()
```



Mmmh, c'est presque linéaire... Ça croît un peu plus vite.

Modélisation

On modélise la séquence des vignettes achetées par une suite de variables aléatoires $X_1, X_2, \dots, X_n$ à valeurs dans $\{1, \dots, M\}$. ## Q3) Hypothèses Quelles sont les hypothèses statistiques sur la suite des variables X_n que l'énoncé suggère?

L'énoncé suggère que les X_n sont des variables indépendantes de loi uniforme.

Q4) Calculs des lois de probabilités

Lorsque la collection comporte déjà $i - 1$ vignettes, on note Y_i le nombre de vignettes à acheter pour avoir une vignette qui n'est pas dans la collection. Donner, en la justifiant, la loi de Y_i (vous pourrez calculer la probabilité $P(Y_i = k)$ et vous en déduirez l'espérance et la variance de Y_i). Les variables Y_i sont-elles indépendantes ? En déduire $\mathbb{E}(T)$ et $\text{Var}(T)$ ainsi que des équivalents quant M est grand.

Étude de Y_i

Lorsque l'on a déjà i vignettes, il reste $M - i$ vignettes à obtenir et donc la probabilité que la prochaine vignette soit correcte est $p_i = \frac{M-i}{M}$. Le nombre de vignettes nécessaires à acheter pour obtenir la suivante suit donc une loi géométrique de paramètre p_i , c'est à dire $P(Y_i = k) = (1 - p_i)^{k-1} p_i$. De ceci, on déduit

$$E(Y_i) = \sum_{k=1}^{\infty} k \cdot P(Y_i = k) = \frac{1}{p_i} = \frac{M}{M-i}.$$

La variance s'obtient après calcul

$$\text{Var}(Y_i) = E(Y_i^2) - E(Y_i)^2 = \frac{1 - p_i}{p_i^2}.$$

Les variables Y_i sont indépendantes car la loi des tirages est uniforme et les tirages sont indépendants.

Étude de T

On a $T(M) = \sum_{i=0}^{M-1} Y_i$, en prenant l'espérance on obtient

$$\text{Esp}(T(M)) = M \left(\frac{1}{M} + \frac{1}{M-1} + \dots + \frac{1}{2} + 1 \right) \simeq M \log M$$

Ou bien un peu plus précisément, en utilisant une bonne approximation de la somme de la série harmonique;

$$\text{Esp}(T(M)) = M(\log M + \gamma + O(1/M))$$

Le log étant petit, c'est pour ça que ça a l'air linéaire mais que mes tentatives pour trouver "la bonne droite" étaient infructueuses...

Faisons le même travail sur $\text{Var}(T)$ car ce n'était visiblement pas clair pour tout le monde. On a

$$\text{Var}(T(M)) = \text{Var}\left(\sum_{i=0}^{M-1} Y_i\right) = \sum_{i=0}^{M-1} \text{Var}(Y_i)$$

car les Y_i sont indépendants deux à deux. On a donc:

$$\text{Var}(T(M)) = \sum_{i=0}^{M-1} \frac{1 - p_i}{p_i^2} = \sum_{i=0}^{M-1} \frac{1}{p_i^2} - \sum_{i=0}^{M-1} \frac{1}{p_i}$$

soit

$$\text{Var}(T(M)) = M^2 \sum_{i=0}^{M-1} \frac{1}{(M-i)^2} - M \sum_{i=0}^{M-1} \frac{1}{M-i} \sim (M^2 \frac{\pi^2}{6} - M \log(M)) \sim \frac{\pi^2}{6} M^2.$$

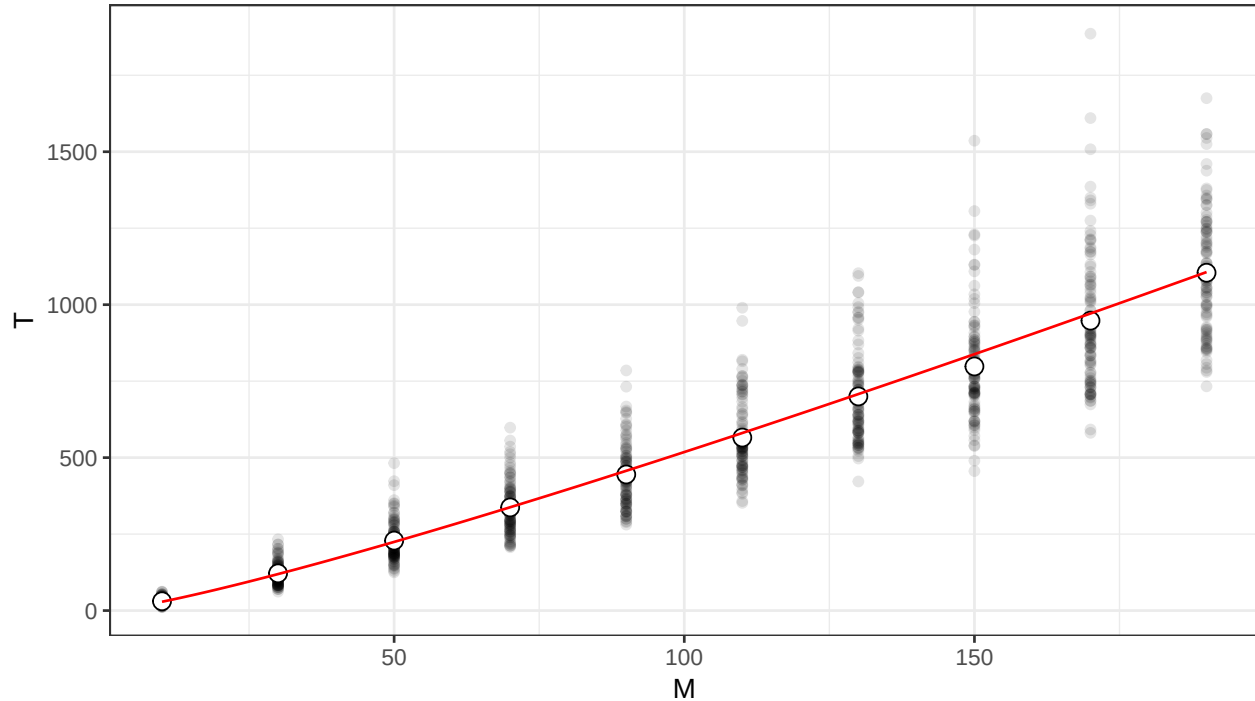
La variance augmente effectivement bien plus vite que l'espérance. Il va falloir faire de plus en plus de simulations pour avoir une estimation correcte.

Synthèse

Q5) Vérification de la cohérence des calculs

Vérifier graphiquement que les résultats obtenus par simulation sont cohérents avec vos calculs.

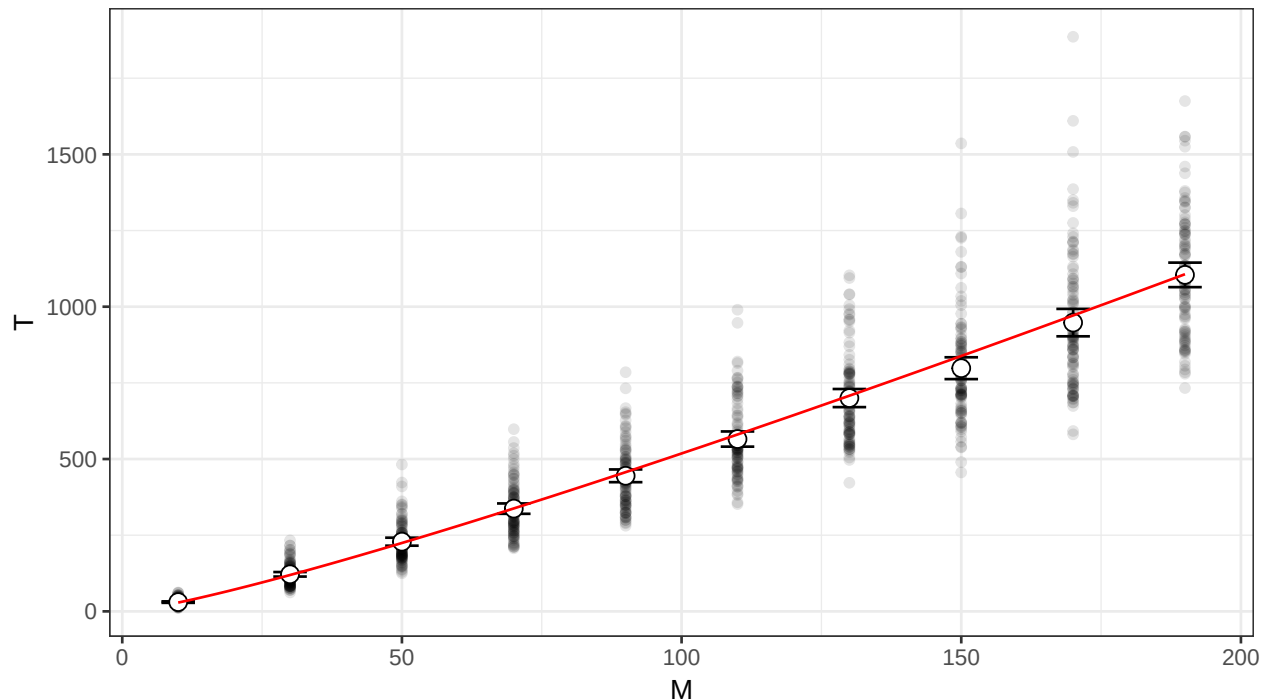
```
E_T <- function(x) {x*(log(x)-digamma(1))}
ggplot(data=df_stat, aes(x=M)) + geom_point(data=df, aes(x=M,y=T),alpha=.1) +
  geom_point(aes(y=mean), size=3, shape=21, fill="white") +
  stat_function(fun = E_T, color="red") +
  theme_bw()
```



Wow, ça marche quand même super bien! :)

Et si on rajoutait les intervalles de confiance, on aurait:

```
ggplot(data=df_stat, aes(x=M)) + geom_point(data=df, aes(x=M,y=T),alpha=.1) +
  geom_errorbar(aes(ymin=mean-se,ymax=mean+se), colour="black", width=6) + geom_point(aes(y=mean), size=3, shape=21, fill="white") +
  stat_function(fun = E_T, color="red") +
  theme_bw()
```



Q6) Et mon petit cousin ?

Au fait, combien votre petit cousin devra-t-il dépenser en moyenne sachant que chaque paquet de 2 euros contient 10 vignettes et que l'album comporte 300 places? Attention, si on veut être précis, ici, il faut pour chaque simulation regarder combien de paquet il faut acheter et en faire la moyenne. C'est différent de faire la moyenne du nombre de cartes nécessaires et de prendre une partie entière supérieure après coup... Comme je ne vais tester que pour un seul album, je peux me permettre d'augmenter le nombre d'échantillons pour avoir une bonne estimation.

```
df=data.frame(M=c(),T=c())
M = 300
for(samples in 1:1000) {
  df = rbind(df,data.frame(M=c(M),T=new_collection(M)))
}
mean(2*ceiling(df$T/10))
```

```
## [1] 379.462
```

Finir l'album coûtera donc en moyenne 379.462 euros... Ça fait quand même cher l'album!

Extension au cas non uniforme

On s'intéresse maintenant au cas où la loi de probabilité d'obtenir chaque vignette n'est plus uniforme.

Q7) Simulation du marchand malhonnête

Adapter votre simulateur afin d'étudier les deux cas suivants: - La probabilité d'obtenir la carte i est égale à $p_i = \frac{1}{\alpha i}$ avec $\alpha = \sum_{k=1}^M \frac{1}{k}$. - La probabilité d'obtenir la carte i est égale à $p_i = \frac{1}{\alpha i^2}$ avec $\alpha = \sum_{k=1}^M \frac{1}{k^2}$. Que remarquez-vous quant à l'influence de cette distribution sur la dépendance de T par rapport à M ?

Fonction de simulation

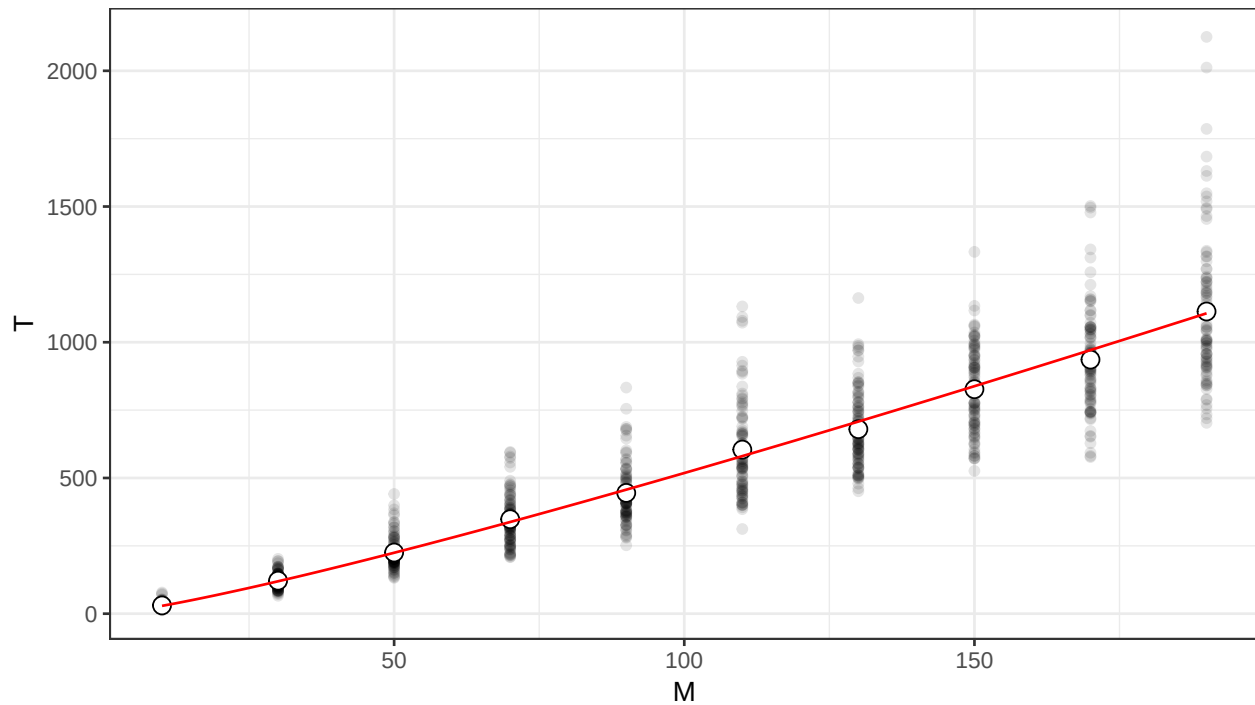
```
new_collection <- function(album_size=100, proba="unif") {  
  switch(proba,  
    "unif" = {  
      p = rep.int(x = 1, album_size);  
    },  
    "harmonic" = {  
      p = 1:album_size;  
      p = 1/p;  
    },  
    "squared" = {  
      p = 1:album_size;  
      p = 1/p**2;  
    }  
  );  
  p = p/sum(p);          # on normalise la proba  
  p = c(0,cumsum(p));    # on somme les valeurs pour retrouver les indices des cartes  
  
  album=rep_len(0, album_size)  
  nb_images = 0;  
  time = 0;  
  while(nb_images<album_size) {  
    time = time + 1;  
    idx = findInterval(runif(1),p); # pourrait aussi s'écrire sum(p<runif(1)) mais findInterval est qua  
    if(album[idx]==0) {  
      album[idx] = 1;  
      nb_images = nb_images + 1;  
    }  
  }  
  time  
}
```

Vérification de la cohérence

Commençons par vérifier que c'est cohérent avec le cas précédent:

```
df=data.frame(M=c(),T=c())  
for(M in seq(10,200,20)) {  
  for(samples in 1:100) {  
    df = rbind(df,data.frame(M=c(M),T=new_collection(M)))  
  }  
}  
df_stat = ddply(df,c("M"),summarise,mean=mean(T),var=var(T),se=2*sqrt(var(T))/sqrt(length(T)))
```

```
E_T <- function(x) {x*(log(x)-digamma(1))}  
ggplot(data=df_stat, aes(x=M)) + geom_point(data=df, aes(x=M,y=T),alpha=.1) +  
  geom_point(aes(y=mean), size=3, shape=21, fill="white") +  
  stat_function(fun = E_T, color="red") +  
  theme_bw()
```

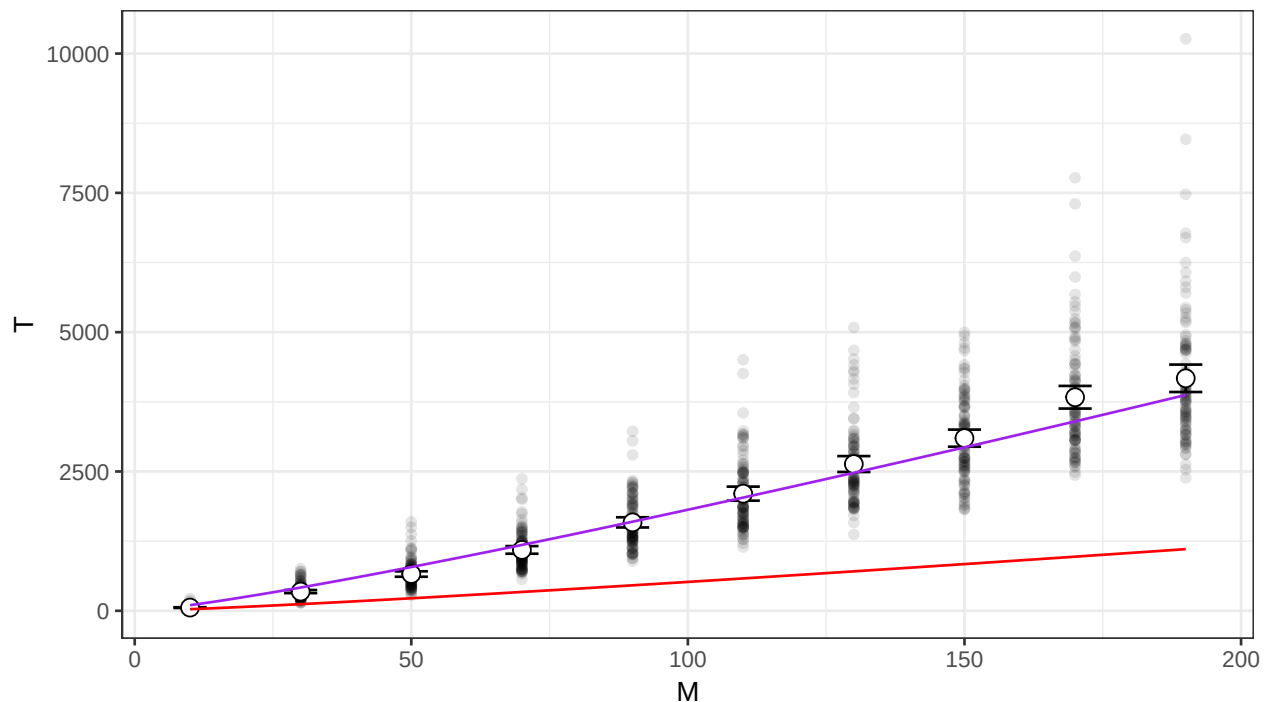


C'est cohérent avec ce qu'on avait avant. Parfait!

Étude du cas harmonique

```
df=data.frame(M=c(),T=c())
for(M in seq(10,200,20)) {
  for(samples in 1:100) {
    df = rbind(df,data.frame(M=c(M),T=new_collection(M,proba = "harmonic")))
  }
}
df_stat = ddply(df,c("M"),summarise,mean=mean(T),var=var(T),se=2*sqrt(var(T))/sqrt(length(T)))

E_T2 <- function(x) {3.5*x*(log(x)-digamma(1))}
ggplot(data=df_stat, aes(x=M)) + geom_point(data=df, aes(x=M,y=T),alpha=.1) +
  geom_errorbar(aes(ymin=mean-se,ymax=mean+se), colour="black", width=6) +
  geom_point(aes(y=mean), size=3, shape=21, fill="white") +
  stat_function(fun = E_T, color="red") +
  stat_function(fun = E_T2, color="purple") +
  theme_bw()
```



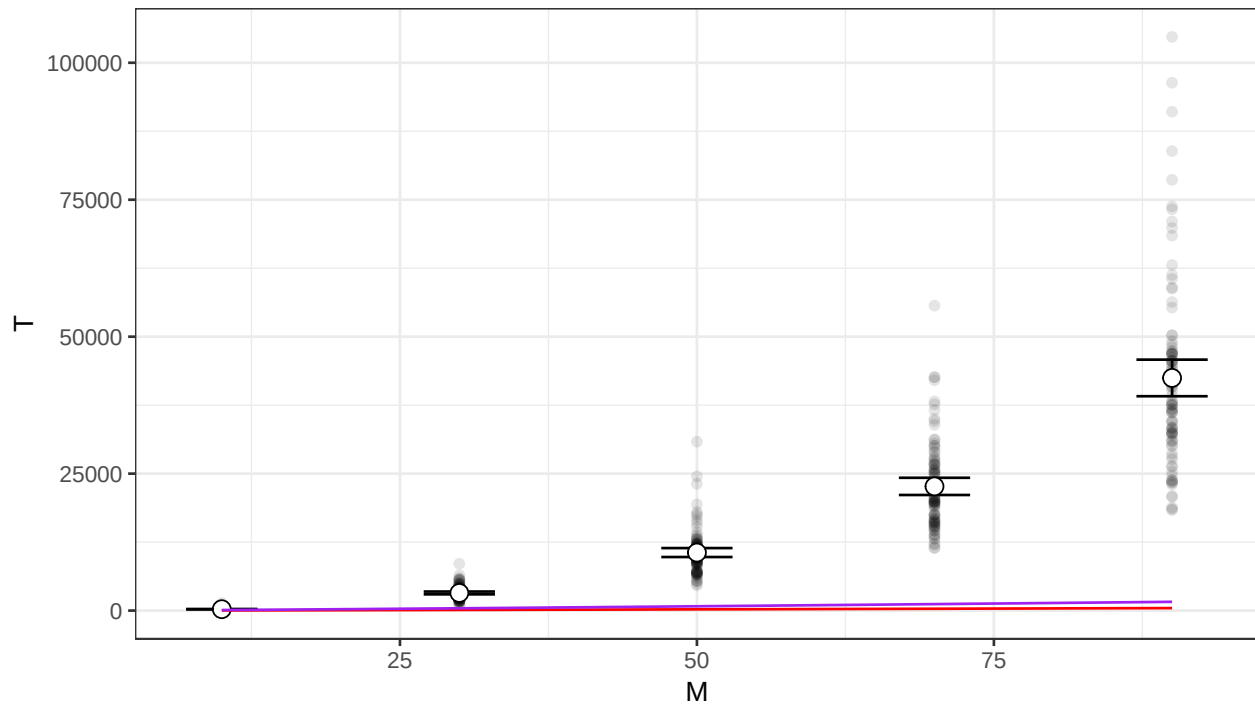
Ah, visiblement, c'est bien plus long (la courbe rouge correspond à la valeur théorique pour le cas uniforme). La courbe violette qui correspond à la courbe rouge multipliée par 3.5 semble indiquer que ce n'est pas juste un facteur multiplicatif. La variance est également bien plus élevée qu'avant et pour avoir un bon estimateur, il faudrait donc faire beaucoup plus d'expériences... :(

Étude du cas quadratique

```
df=data.frame(M=c(),T=c())
for(M in seq(10,100,20)) {
  for(samples in 1:100) {
    df = rbind(df,data.frame(M=c(M),T=new_collection(M,proba = "squared")))
  }
}
df_stat = ddply(df,c("M"),summarise,mean=mean(T),var=var(T),se=2*sqrt(var(T))/sqrt(length(T)))
```

Aouch, déjà, on sent que c'est beaucoup plus long à simuler. C'est pour ça que je diminue le nombre de valeurs pour M et même comme ça il faut être patient.

```
ggplot(data=df_stat, aes(x=M)) + geom_point(data=df, aes(x=M,y=T),alpha=.1) +
  geom_errorbar(aes(ymin=mean-se,ymax=mean+se), colour="black", width=6) +
  geom_point(aes(y=mean), size=3, shape=21, fill="white") +
  stat_function(fun = E_T, color="red") +
  stat_function(fun = E_T2, color="purple") +
  theme_bw()
```



Wow et là, ça explose complètement! C'est assez logique en même temps, les probabilités d'avoir les dernières cartes sont extrêmement faibles...

Q8) Le cas d'une distribution exponentielle

Selon vous, que se passera-t-il pour une simulation avec $p_i = \frac{1}{\alpha 2^i}$ et $\alpha = \sum_{k=1}^M \frac{1}{2^k}$?

La probabilité d'avoir la carte M est donc de l'ordre de $1/2^M$, ce qui signifie qu'on l'obtient en moyenne au bout de 2^M tirages! Pour $M = 200$, ça fait 1.606938×10^{60} donc notre simulation ne serait pas prête de terminer.